THE ARCHITECT'S FIELD NOTES

BEHAVIORAL INTERVIEW STUDY GUIDE / 01 OF 07

System Design Interview — Architecture Fundamentals Primer

THINK CLEARLY. DECIDE WITH EVIDENCE. DELIVER WITH CONFIDENCE.

Evidence-based STAR answers, senior-level judgment, and focused interview rehearsal.

PREPARED FOR SENIOR / PRINCIPAL SOLUTION ARCHITECT INTERVIEWS

READ · REFLECT · PRACTICE

01



YOUR STUDY COMPANION

American Express · Solution Architect · Module 1

Purpose: Learn the essential concepts and architectural decisions before studying the system design examples.

This primer focuses on **why we choose a design**, not memorizing definitions. We use payment processing as the running example.

NAVIGATION

Inside this guide

01	1. The most important principle
02	2. Start with requirements
03	3. Build the simplest correct architecture
04	4. Capacity estimation — enough to guide decisions
05	5. API and service communication
06	6. Database selection — SQL vs. NoSQL
07	7. Reliability patterns you must recognize
08	8. Distributed transactions: the highest-value patterns
09	9. Scaling: how to reason about bottlenecks
10	10. Message queue vs. event streaming
11	11. Single-region vs. multi-region architecture
12	12. Security is an architectural constraint
13	13. Observability and operational readiness
14	14. Architecture choice cheat sheet
15	15. Put everything together: payment API design
16	16. What distinguishes a Solution Architect answer
17	17. The mental model to remember
18	Official references for further reading

CHAPTER

1. The most important principle

System design is about making trade-offs under constraints.

There is no universally best database, communication protocol, architecture, or cloud service.

Consider three requirements:

Requirement	Architectural implication
Payment must not be processed twice	Idempotency, uniqueness, transaction state
Dashboard must display millions of transactions quickly	Read replicas, indexing, caching, analytical stores
Customer must receive a payment notification	Asynchronous messaging, retries

All three involve transaction data, but they need different solutions.

A common interview mistake is starting with technology:

> "I'll use microservices, Kafka, Redis, and PostgreSQL."

A better approach is:

> "We need transactional correctness for payments, so I'll start with a relational database and atomic state transitions. Notifications can be asynchronous because they do not affect the payment decision."

The requirement should justify the technology, not the other way around.

CHAPTER

2. Start with requirements

Before designing, separate three things.

A. Functional requirements

What must the system do?

For payment authorization:

- Accept an authorization request.
- Validate the request and customer/payment context.
- Check applicable risk, account, and authorization rules.
- Produce an authorization decision.
- Store the transaction state.
- Support duplicate requests and status lookup.

B. Non-functional requirements (NFRs)

How well must the system operate?

NFR	What it means	Design impact
Latency	How quickly must the system respond?	Avoid slow calls in critical path
Availability	How often must service be usable?	Redundancy, failover
Scalability	Can it handle growing traffic?	Horizontal scaling, partitioning
Consistency	How fresh and correct must data be?	Transactions, synchronization
Durability	Must data survive failures?	Durable storage, recovery
Security	Who can access sensitive data?	IAM, encryption, isolation
Reliability	Can it operate correctly despite failures?	Retries, idempotency, recovery
Cost	What resources can the business afford?	Simplicity, managed services, capacity decisions
Maintainability	Can teams change it safely?	Clear boundaries, testing, observability

C. Business invariants

An **invariant** is a rule that must remain true even when requests arrive concurrently or systems fail.

Examples:

- The same logical payment must not create two successful charges.
- A financial ledger must remain balanced.
- A ticket cannot be allocated to two customers.
- A completed refund cannot exceed the eligible amount.

These rules are more important than individual technologies.

Design rule: Identify the invariants first. Then determine where they must be enforced.

CHAPTER

3. Build the simplest correct architecture

Start with a baseline, not a large distributed system.

```
Client
  |
  v
API Service
  |
  v
Database
```

This is a valid starting point.

Why?

- Fewer moving parts.

- Simple deployment and debugging.
- Database transactions can enforce critical rules.
- Lower infrastructure complexity.

Now introduce requirements that force changes.

If you need...	Consider...	Why
Higher availability	Redundant application instances and load balancing	Survive individual instance failures
Faster repeated reads	Cache for data that can safely be stale	Reduce repeated database work
Slow noncritical work	Asynchronous processing with a queue	Keep background work off the critical path
More database capacity	Query optimization, replicas, partitioning, then possibly sharding	Address measured bottlenecks
Independent business ownership	Services separated along business boundaries	Support independent change and operations

Key lesson: Every extra component adds operational cost and possible failure modes. Add it to solve a demonstrated requirement.

CHAPTER

4. Capacity estimation — enough to guide decisions

You do not need perfect calculations. You need estimates that reveal likely bottlenecks.

Four quantities matter

Metric	Why it matters
Average requests per second (RPS)	Normal application load
Peak RPS	Capacity needed during spikes
Data volume and retention	Storage size and growth
Read/write ratio	Caching, replication, and database selection

Suppose a payment system expects a **hypothetical** peak of 10,000 requests per second.

Questions to derive:

- Can one service instance process this traffic while meeting latency targets?
- How many database operations does each request generate?
- Are all requests writes, or can some use cached/read-only data?
- Will the database sustain the resulting write throughput?
- How much historical data must stay in the primary database?
- What happens if traffic doubles or a region fails?

Little's Law

For a stable system:

$$ L = \lambda W $$

Where:

- L = average concurrent requests.
- λ = average request arrival rate.
- W = average time each request spends in the system.

This helps estimate concurrency and resource requirements.

A critical distinction: **Average capacity does not guarantee acceptable tail latency.** Even if most requests are fast, the slowest few percent can severely affect user experience.

So we normally discuss p95 or p99 latency, not only the average.

The figures above are illustrative, not American Express production requirements.

CHAPTER

5. API and service communication

Three communication approaches appear repeatedly in system design.

Approach	Best suited for	Trade-off
REST/HTTP	Public APIs, CRUD, partner integration	Simple, widely supported, but commonly synchronous
gRPC	Internal service-to-service requests	Efficient contracts, but more tooling and integration constraints
Events/messages	Background processing and decoupled services	Better isolation, but eventual consistency and operational complexity

Synchronous communication

```
Payment API ---> Fraud Service
                 |
Payment API <--- Fraud Result
```

The caller waits for a response.

Use when: The result is required before continuing.

For example, payment authorization may need a real-time risk decision.

Risk: If the dependency is slow, the caller may also become slow.

Asynchronous communication

```
Payment Service
       |
       v
```

```
    Message Broker
        |
        +---> Notification Service
        |
        +---> Analytics Service
```

The publisher does not wait for all consumers to finish.

Use when: Work can happen independently or later.

Trade-off: The system becomes eventually consistent, and the message may be delayed, duplicated, or require recovery.

API design considerations

A good enterprise API needs more than endpoints.

Concern	Recommended approach
Contract	Define requests, responses, errors, versions
Authentication	OAuth/OIDC or suitable service authentication
Authorization	Enforce access to resources and actions
Retries	Document retryable errors and backoff
Duplicate requests	Use idempotency keys for operations needing them
Long-running requests	Return an operation ID and expose status
Compatibility	Prefer backward-compatible API evolution

For payment creation, an idempotency key might look like:

```
POST /payments
Idempotency-Key: order-123-payment-1
```

If the client retries the same logical operation, the system must not create another payment.

Use a durable uniqueness constraint and persist the request identity, parameters, and result/state. A Redis cache alone is not a reliable financial deduplication boundary.

CHAPTER

6. Database selection — SQL vs. NoSQL

Do not start by asking which database is faster.

Start with:

What data is stored? What queries run? What correctness guarantees are required?

Relational databases

Examples: PostgreSQL, MySQL, SQL Server, Oracle.

Prefer relational databases when:

- Multiple records must change atomically.
- The data has well-defined relationships.
- Constraints are important.
- Queries need joins and flexible filtering.
- Financial correctness matters.

Payment example: Creating ledger entries and updating a transaction record must satisfy accounting rules.

NoSQL databases

NoSQL is a broad category, not one database architecture.

Category	Typical use
Key-value	Fast lookup, sessions, counters
Document	Flexible structured documents
Wide-column	Large distributed workloads organized around access patterns
Graph	Relationship traversal, connected entities

NoSQL does not automatically mean weak consistency. Guarantees depend on the database and configuration.

Example: Customer preference documents may suit a document database, while ledger entries may suit a transactional relational database.

ACID transactions

ACID describes four properties:

Property	Meaning
Atomicity	All changes succeed or none commit
Consistency	Transaction preserves defined database rules
Isolation	Concurrent transactions behave according to the chosen isolation level
Durability	Committed data survives supported failure scenarios

A critical nuance: **ACID does not automatically prevent every business race condition.**

For example, two transactions might both observe available inventory and attempt to reserve it. The solution may need a conditional update, row lock, unique constraint, or stronger isolation.

Strong vs. eventual consistency

Strong consistency: Reads observe the required latest committed state under the system's stated consistency model.

Example: Preventing two transactions from spending the same reserved funds.

Eventual consistency: Replicas or downstream projections may temporarily differ but converge when updates stop and delivery succeeds.

Example: Transaction analytics updating after a payment event.

Rule: Use strict correctness where business invariants require it. Allow eventual consistency for downstream read models where delay is acceptable.

CHAPTER

7. Reliability patterns you must recognize

A distributed system must expect network failures, process crashes, dependency outages, and ambiguous results.

Timeouts

Every remote request needs a bounded wait.

Without timeouts, threads or connections can remain occupied indefinitely.

Design choice: Assign an overall request deadline and smaller budgets to dependencies.

Retries with backoff and jitter

Retry transient failures, not every error.

- **Backoff:** Increase the delay between retries.
- **Jitter:** Randomize the delay to prevent many clients retrying together.
- **Retry budget:** Limit retry attempts to prevent overload.

A retry is safe only when the operation has appropriate idempotency or duplicate protection.

Circuit breaker

When a dependency repeatedly fails, stop sending requests temporarily.



This prevents cascading failures.

But a circuit breaker is not a correctness mechanism. If a payment provider is unavailable, do not automatically approve a transaction merely because the dependency failed.

Bulkhead

Isolate resource pools so one unhealthy workload does not consume everything.

Example: Analytics jobs must not exhaust the same database connection pool used by payment authorization.

Dead-letter queue (DLQ)

Messages that repeatedly fail processing are moved to a separate location for inspection and recovery.

A DLQ is not a substitute for handling poison messages, alerting, or replay procedures.

Disaster recovery: RTO and RPO

Term	Meaning
RTO	Maximum acceptable recovery time
RPO	Maximum acceptable data loss measured in time

A business requiring a very small RPO may need stronger replication and recovery mechanisms than nightly backups.

CHAPTER

8. Distributed transactions: the highest-value patterns

This section matters for payments, checkout, transfers, and enterprise workflows.

Problem: A business transaction spans services

Suppose an order requires:

```
Create Order
     |
     v
Reserve Inventory
     |
     v
Process Payment
     |
     v
Confirm Order
```

Each service owns a separate database.

A normal database transaction cannot simply guarantee atomic changes across all these independent systems.

Saga pattern

A Saga coordinates multiple local transactions and defines recovery actions.

If payment fails, release the inventory reservation and cancel the order.

Two common choices:

Pattern	Best when
Choreography	Few participants, simple event-based flow
Orchestration	Complex workflows requiring central visibility and control

Important: Compensation is not the same as database rollback. A refund, for example, is a new financial operation that reverses an earlier business effect.

Transactional outbox

Consider this sequence:

1. Payment state is committed to the database.

2. The service crashes before publishing `PaymentCompleted`.
3. Downstream systems never receive the event.

This is the **dual-write problem**.

Solution:

```
Database Transaction
+------------------+
| Update Payment   |
| Insert Outbox    |
+------------------+
         |
      Commit
         |
         v
  Outbox Publisher
         |
         v
    Message Broker
```

The payment update and outbox event are committed atomically in the same database.

A publisher then delivers the event to the broker.

Trade-off: Delivery can still be duplicated. Consumers must be idempotent.

Exactly-once vs. effectively-once

Do not casually promise end-to-end exactly-once execution.

In practice, systems often combine:

- At-least-once message delivery.
- Unique transaction identifiers.
- Idempotent consumers.
- Atomic database updates.
- Reconciliation for uncertain outcomes.

Together, these can deliver the required business effect once even when messages or requests are retried.

CHAPTER

9. Scaling: how to reason about bottlenecks

Horizontal vs. vertical scaling

Vertical scaling: Increase CPU, memory, or capacity of one instance.

Horizontal scaling: Add instances and distribute workload.

For stateless API services, horizontal scaling is usually straightforward when shared state lives outside the instances.

```
              Load Balancer
```

```
              /   |   \
             v    v    v
          API1  API2  API3
             \   |   /
              Database
```

But notice the remaining bottleneck: the database.

Cache vs. read replica vs. sharding

Technique	Solves	Does not automatically solve
Cache	Repeated expensive reads	Correctness of writes
Read replica	Read load and some locality needs	Primary write bottleneck
Sharding	Data/write volume across nodes	Cross-shard transactions
Partitioning	Data organization and query efficiency	Every underlying capacity limit
CDN	Static and cacheable edge delivery	Transaction processing

Cache-aside pattern

```
        Read Request
             |
             v
          Cache
         /     \
       Hit     Miss
        |        |
      Return   Database
                 |
              Populate
                 |
              Return
```

The difficult part is not reading the cache. It is **invalidation and freshness** .

For financial balances, a stale cache can be dangerous if used for authorization decisions.

Prefer an authoritative transactional state when making correctness-critical decisions. A cache may still support nonauthoritative display or carefully controlled reads.

Sharding

Sharding splits data across multiple independent storage partitions.

Example:

```
Account ID
    |
    v
Shard Router
    |
    +---> Shard A
    +---> Shard B
    +---> Shard C
```

A good shard key should distribute load while keeping common operations efficient.

Risk: Transfers involving accounts on different shards become more complex. Hot accounts can also create uneven traffic.

Do not introduce sharding merely because a system is described as large.

CHAPTER

10. Message queue vs. event streaming

These are related, but they solve different needs.

Dimension	Work queue	Event stream
Main purpose	Distribute work	Record and distribute events
Consumption	Workers process jobs	Consumers process streams
Replay	Depends on queue design	Commonly supported via retained logs
Typical example	Send email	Process transaction activity
Technologies	SQS, RabbitMQ	Kafka, similar event log platforms

Kafka concepts

- **Topic:** Named event stream.
- **Partition:** Unit of parallelism and ordered storage.
- **Producer:** Writes records.
- **Consumer group:** Coordinates consumption among members.
- **Offset:** Position within a partition.

Kafka preserves record order within a partition, not necessarily across an entire topic.

Example: Partition fraud-related events by account ID when per-account ordering is required.

This creates ordering within an account's partition, but a very active account could become a hot partition.

Decision principle: Choose a work queue for job distribution. Choose an event stream when replay, multiple independent consumers, or ordered event history is a central requirement. Both categories have overlapping capabilities.

CHAPTER

11. Single-region vs. multi-region architecture

There are three important starting designs.

Architecture	Advantage	Main challenge
Single region, multi-AZ	Lower complexity, zone resilience	Regional outage

Active-passive regions	Disaster recovery with simpler write ownership	Failover and recovery time
Active-active regions	Regional availability and locality	Conflicts, coordination, replication complexity

For financial transactions, active-active writes can be particularly difficult.

Example:

- Region A sees an available balance of $100.
- Region B sees the same balance.
- Both approve spending $80.

If both independently make decisions using stale replicated data, the account can be overspent.

Better options: Single-writer ownership per account, strongly coordinated writes, or business-specific reservation mechanisms.

These choices must be derived from latency, availability, data-loss, and consistency requirements.

CHAPTER

12. Security is an architectural constraint

For financial services, security cannot be added at the end.

Essential concepts

Concept	What it means
Authentication	Verify identity
Authorization	Check permitted actions
Least privilege	Grant only necessary access
Encryption in transit	Protect data over networks
Encryption at rest	Protect stored data
Tokenization	Replace sensitive values with tokens
HSM	Hardware-backed protection and use of cryptographic keys
Audit logging	Record relevant security and business actions
Zero trust	Do not grant implicit trust based on network location

NIST's zero-trust guidance emphasizes protecting resources and verifying access rather than trusting a network boundary.

PCI DSS in payment designs

PCI DSS provides security requirements for systems that store, process, transmit, or can affect payment card account data.

For a payment architecture:

- Minimize services that handle raw card data.
- Use tokenization where appropriate.

- Restrict access to sensitive data.
- Keep secrets out of application code.
- Prevent sensitive account data from appearing in ordinary logs.
- Apply approved key management and retention controls.
- Treat compliance scope as a design consideration.

Important: Tokenization alone does not guarantee that a service is outside PCI scope. The complete data flow and connected systems matter.

CHAPTER

13. Observability and operational readiness

A design is incomplete if the team cannot detect or recover from problems.

Three main signals

Signal	Purpose
Metrics	Measure behavior and trends
Logs	Explain individual events and errors
Traces	Follow requests across services

SLI, SLO, SLA

- **SLI:** What we measure, such as payment authorization latency.
- **SLO:** Internal reliability or performance target.
- **SLA:** Contractual service commitment.

For payment processing, monitor more than HTTP success rates.

Useful business and technical signals include:

- Authorization decision latency.
- Timeout and unknown-outcome rates.
- Duplicate attempts prevented.
- Database transaction failures.
- Event delivery lag.
- Reconciliation exceptions.
- Dependency error rates.
- Transaction state transitions.

Key point: An API returning HTTP 200 does not necessarily mean the financial workflow completed correctly.

CHAPTER

14. Architecture choice cheat sheet

This is the most valuable section for quick reference.

Situation	Start with	Why
Financial transaction correctness	Relational DB + constraints	Atomic updates and invariants
High read traffic	Cache or read replicas	Reduce repeated database work
High write volume	Optimize, partition, then shard if needed	Scale actual bottleneck
Immediate decision required	Synchronous request	Caller needs a result
Background task	Queue	Decouple execution
Multiple independent event consumers	Event stream/pub-sub	Independent processing
Cross-service business workflow	Saga	Coordinate local transactions
Database update + event publication	Transactional outbox	Avoid lost dual writes
Duplicate request risk	Durable idempotency record	Prevent duplicate effects
Inventory contention	Conditional write/locking	Prevent double allocation
Slow dependency	Timeout + circuit breaker	Limit cascading failures
Transient dependency error	Bounded retry + jitter	Recover without overload
Complex enterprise migration	Strangler pattern	Migrate incrementally
Regional disaster recovery	Active-passive initially	Simpler write ownership
Global concurrent writes	Explicit conflict/coordination design	Preserve invariants
Sensitive payment data	Restricted security boundary	Minimize exposure and scope

Notice that these are **starting choices**, not absolute rules.

CHAPTER

15. Put everything together: payment API design

Here is how these principles combine in one architecture.



```
Control    State DB Processor
             |
             v
         Outbox Table
             |
             v
        Event Publisher
             |
             v
         Message Broker
         /     |      \
        v      v       v
 Notification Audit  Reconciliation
```

This is a conceptual architecture, not a complete card-network authorization design.

Deriving the choices

1. Payment Service: Owns payment state transitions because the business needs one clear place to enforce lifecycle rules.

2. Idempotency Control: Prevents the same logical request from creating multiple operations. The enforcement uses durable state and unique constraints.

3. Payment State Database: Holds authoritative operation status. Transactional writes protect state transitions.

4. External Processor: Performs the required external payment operation. Timeouts can leave an uncertain result, which must be tracked rather than assumed to be a failure.

5. Transactional Outbox: Records committed state-change events without losing them during a crash.

6. Message Broker: Separates noncritical downstream processing from the synchronous payment path.

7. Notification Service: Can retry delivery without repeating the payment.

8. Audit and Reconciliation: Provide traceability and detect differences between internal and external records.

This architecture still needs decisions about authentication, card-data boundaries, provider integration, failover, settlement, monitoring, and operational ownership.

That is where the full Card Payment Authorization design will go deeper.

CHAPTER

16. What distinguishes a Solution Architect answer

A developer-focused design may stop after selecting services, databases, and communication patterns.

A Solution Architect should also address:

Dimension	What to explain
Business alignment	Which requirements drive each choice?
Integration	How does the solution connect with existing systems?
Boundaries	Which service owns each capability and its data?
Migration	How do we move safely from the current system?

Governance	How are APIs, security, and technology choices standardized?
Resilience	What happens when components or regions fail?
Operations	Who supports the system, and how is it observed?
Cost	Is the complexity justified by the business need?
Evolution	Can the architecture change without major disruption?

For example, replacing a mainframe transaction system with microservices is not necessarily an improvement.

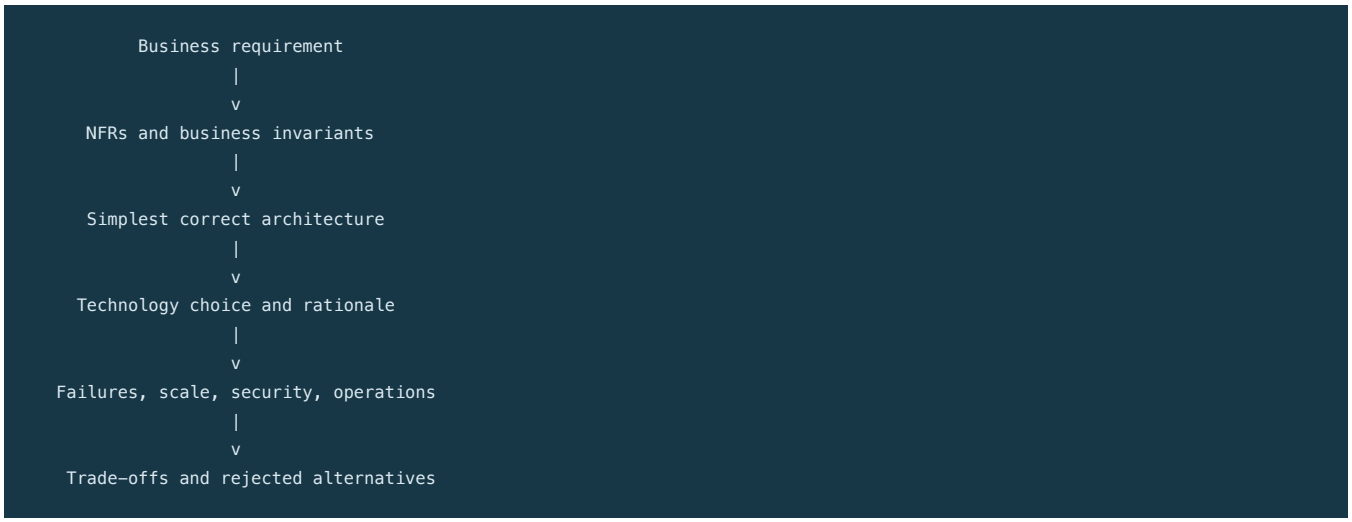
A more appropriate approach might be to place an API layer in front of the existing system, migrate capabilities gradually, and preserve a single authoritative transaction owner during migration.

This is the reasoning behind incremental modernization patterns such as the Strangler Fig pattern.

CHAPTER

17. The mental model to remember

For every system design, follow this sequence:



The central idea is:

Do not say "I will use Kafka" before explaining why asynchronous events, replay, partitioned processing, or independent consumers are needed.

Do not say "I will use microservices" before establishing why separate deployment, scaling, or ownership boundaries are worth the added complexity.

Do not say "I will use multi-region active-active" before deciding how conflicting writes and financial correctness will be handled.

That reasoning is what we will apply repeatedly throughout the selected designs.

Next: Card Payment Authorization — full end-to-end system design, derived from requirements through final architecture, including failures and alternatives.

CHAPTER

Official references for further reading

These are optional. The primer is designed to be useful on its own.

- PostgreSQL — Transaction Isolation
- Apache Kafka — Documentation
- NIST SP 800-207 — Zero Trust Architecture
- PCI Security Standards Council — Official site
- AWS Well-Architected Framework
- AWS Prescriptive Guidance — Cloud Design Patterns